Learning Transition Paths in Turbulent Domains with Neural Operators

by

Thomas Higham

MA4K9 Dissertation

Submitted to The University of Warwick

Mathematics Institute

April, 2025

Abstract

Operator learning is an extension of techniques from machine learning to function spaces. In particular, neural operators can be used to learn solutions to a class of partial differential equations (PDEs). Given input parameters corresponding to a particular PDE in this class, it can evaluate a predicted solution to this equation anywhere in the domain. Transition paths concern the probability density of the pathway a stochastic process makes between two sets, moving by diffusion and advection. The corresponding probability density functions can be obtained by solving Fokker-Planck equations. In this paper we train a Fourier neural operator to predict the product of solutions to these equations in order to determine transition paths on turbulent domains. The main contributions of this work are; equation (18) which simplifies the probability density function when the stochastic process has constant diffusion; Algorithms 1 and 2 which generate C^2 periodic domains and turbulence respectively; and Theorem 32 which shows how the steady advection-diffusion equation is changed under a coordinate transform of the C^2 domain to the square $[0,1]^2$. The test results indicate that FNOs show promise at predicting the shapes of transition pathways but the relative test error is too high for accurate prediction. Python code for this paper is freely available here.

Contents

1 Introduction **1**

2 Operator Learning **4**
2.1 Supervised Learning 4
2.2 Training and Testing 5
2.3 Fourier Neural Operator 6

3 Meet the Fokkers **7**
3.1 The Forward Committor Function 7
3.2 The Steady Probability Density Function 8
3.3 The Backwards Committor Function 9

4 Generating Training Data **9**
4.1 Generating Domains 9
4.2 Generating a Turbulent Velocity Field 11
4.2.1 Turbulence Theory 11
4.2.2 Turbulence Algorithm 14
4.3 Finite Difference Solvers 15
4.3.1 Forward Committor Solver 19
4.3.2 Backward Committor Solver 19

5 Training the Neural Operator **20**
5.1 Square Domain Results 22
5.2 Curved Domain Results 23

6 Conclusion and Further Work to be Done **27**

7 References **28**

8 Appendix - Derivation of Infinitesimal Generator **29**

1 Introduction

Operator learning is an exciting blend of PDEs and machine learning, first introduced in a paper by Lu et al in 2019 [14]. It extends concepts from neural networks and enables accurate approximation of non-linear functions. The power of neural operators is that they can learn solutions to families of PDEs and, with the appropriate training data, can predict solutions to PDEs that we might not be able to solve computationally. Once trained a neural operator can predict solutions given input data, such as the coefficients to a PDE, within seconds and can be used to find solutions anywhere on a domain even if trained on a finite discretisation. The key aims in operator learning research at the moment are to increase the accuracy of prediction and the ability to generalise to out-of-distribution events.

Transition paths are based on a simple question: if you have a particle that can move between two disjoint sites, A and B, through a combination of random movement and advection from a velocity field then what is the most likely pathway that it will take. This kind of movement is often modelled as a Markov process $\{X_t\}_{t\in T} \subseteq \Omega \subset \mathbb{R}^n$ governed by a stochastic differential equation (SDE) of the form

$$\mathrm{d}X_t = b(X_t, t)\ \mathrm{d}t + \sigma(X_t, t)\ \mathrm{d}W_t, \tag{1}$$

where W_t is a standard Brownian motion in $\mathbb{R}^n$, b is vector-valued and σ matrix-valued (see Pavliotis [17] for background on stochastic processes). We often call b the drift coefficient and σ the diffusion coefficient. In this paper we consider stochastic processes $\{X_t\}_{t\in T}$ belonging to a 2-D domain Ω that is vertically periodic and bounded on the left and right by sets A and B; the top and bottom of the domain are parallel and equal lines, and the sides of the domain can be thought of as a regular deformation of the sides of a rectangle. Figure 1 shows an example domain. This domain is chosen to model a turbulent flow in a fluid. We want to predict the most likely transition path that a particle starting in set A will take to get to set B. We simplify the SDE to

$$\mathrm{d}X_t = b(X_t)\mathrm{d}t + \sqrt{2}I_2\mathrm{d}W_t, \tag{2}$$

by assuming that the turbulent velocity field, b, has reached an equilibrium state and letting $\sigma(X_t) = \sqrt{2}I_2$, where I_2 is the identity matrix in 2-D. Each random variable X_t has a probability density ρ that evolves according to the Fokker-Planck equation

$$\partial_t \rho = \nabla \cdot (-b(\boldsymbol{x})\rho + \nabla\rho). \tag{3}$$

Furthermore each SDE (1) defines a Markov process with infinitesimal generator

$$Lf = b \cdot \nabla f + \Delta f. \tag{4}$$

See Appendix 8 for details on the derivation. When $\partial_t \rho = 0$ in (3) we call the solution $\rho(x)$ the steady probability density function.

Given a set S and a time t we define the hitting times

$$\tau_S := \inf\{s > 0 \mid X_s \in S\}, \quad \tau_S^+ := \inf_{s>0}\{s \mid X_{t+s} \in S\}, \quad \tau_S^- := \sup_{s<0}\{s \mid X_{t+s} \in S\},$$

where the latter two expressions needs to be conditioned on $X_t = \boldsymbol{x} \in \Omega$ (the bold face emphasizes that this is 2-D vector). These represent the first time a stochastic process reaches the set S, the next time we reach S in the future, and the last time we hit S in the past.

We are interested in the probability that the stochastic process next visits B rather than A, and the probability that it last visited A rather than B. We define the *forward and backward committor functions*, q^+ and q^-, as

$$q^+(x) = P(\tau_B^+ < \tau_A^+ \mid X_t = \boldsymbol{x}), \quad q^-(x) = P(\tau_B^- < \tau_A^- | X_t = \boldsymbol{x}). \tag{5}$$

These are conditional probability densities which can be found by solving the discrete Dirichlet problems

$$\begin{cases} Lq^+(\boldsymbol{x}) = 0 & \forall \boldsymbol{x} \in \Omega, \\ q^+(\boldsymbol{x}) = 0, & \forall \boldsymbol{x} \in \partial A, \\ q^+(\boldsymbol{x}) = 1, & \forall \boldsymbol{x} \in \partial B, \end{cases}$$

and

$$\begin{cases} \bar{L}q^-(\boldsymbol{x}) = 0, & \forall \boldsymbol{x} \in \Omega, \\ q^-(\boldsymbol{x}) = 1, & \forall \boldsymbol{x} \in \partial A, \\ q^-(\boldsymbol{x}) = 0 & \forall \boldsymbol{x} \in \partial B, \end{cases}$$

where $\bar{L}$ is the generator of the reverse Markov process given by

$$\bar{L}q^- := \Delta q^- + b^R \cdot \nabla q^-, \quad \text{with } b^R(\boldsymbol{x}) = -b(\boldsymbol{x}) + \frac{2}{\rho(\boldsymbol{x})}\nabla \cdot (a(\boldsymbol{x})\rho(\boldsymbol{x})), \tag{6}$$

(see Section 3.3 for where this comes from). Of particular interest is the probability density of *reactive trajectories*

$$\rho_{\text{react}}(\boldsymbol{x}) = q^-(\boldsymbol{x})\rho(\boldsymbol{x})q^+(\boldsymbol{x}) \tag{7}$$

which concerns the probability of coming from A last, being at $\boldsymbol{x}$, and going to B next. This quantity peaks at regions where trajectories transitioning from A to B spend most of their time. Therefore ρ_{react} can help us to identify dynamical bottlenecks in a system.

The aim of this paper is to see if operator learning can learn the probability density of reactive trajectories. This is an interesting application of neural operators where we try to predict the product of the solutions to several related PDEs, as opposed to a solution

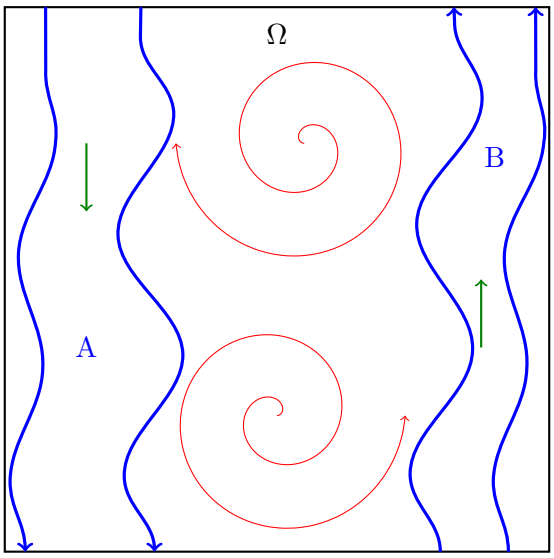

Figure 1: Turbulent flow diagram. The domain Ω lies between sets A and B

to one class of PDE. We train a neural operator to predict the "coefficient to solution" map meaning that it predicts the value of ρ_{react} at a point $\boldsymbol{x}$ given a grid of values of b. Furthermore to make the problem more realistic we investigate whether neural operators can generalise to different domain shapes: normally they are just applied to squares. For such domains we try to learn a "coefficient and domain to solution" map where, in addition to b, values of a transformation f^{-1}, that maps the square $[0,1]^2$ to the domain Ω, are given. This map is detailed in Section 4.3.

There are physically motivated applications of predicting transition paths on non-square turbulent domains. Firstly, one could predict the transition path of an oil spill between two locations in the ocean, perhaps the site of the spill and a coral reef or beach. Neural operators are quick at predicting solutions once trained so we could employ neural operators over a range of input parameter uncertainty, from physical data, to inform response strategies to the oil spill. Another possible application could be in solving magnetohydrodynamic equations in 3-D that model the movement of plasma. Turbulent magnetic fields can lead to turbulent movement of plasma. An example use is in predicting the transition paths of solar winds and where they are likely to hit the Earth. This is useful as solar winds can cause geomagnetic storms which can damage main electrical grids and cause power outages.

We start by introducing neural operators formally in Section 2 and specifying the Fourier neural operator architecture. Then we discuss the committor functions, q^+ and q^- defined in (5), in Section 3. In Section 4 we discuss algorithms for generating domains Ω, synthetic turbulent velocity fields, and finite difference solvers for generating training data. An introduction to turbulence theory is given to justify the turbulence algorithm. In Section 5 we discuss training the neural operator and the results. In Section 6 we discuss how well

Fourier neural operators predict transition paths and list some future work that can be done.

The code used in this paper is generated in Python and freely available in the following Github repository.

2 Operator Learning

The aim of operator learning is to learn the mapping between two infinite dimensional function spaces $\mathcal{U}$ and $\mathcal{V}$ [1]. While the theory is conceived on a continuum level, the operator is implemented as a discrete approximation. We use the notation of Kovachki et al. [9] and consider separable Banach spaces of vector valued functions:

$$\begin{aligned} \mathcal{U} &= \{u : \mathfrak{D}_x \to \mathbb{R}^{d_i}\}, \quad \mathfrak{D}_x \subseteq \mathbb{R}^{d_x}, \\ \mathcal{V} &= \{v : \mathfrak{D}_y \to \mathbb{R}^{d_o}\}, \quad \mathfrak{D}_y \subseteq \mathbb{R}^{d_y}. \end{aligned}$$

Operator learning generalises neural networks, which learn a mapping between two finite dimensional vector spaces. Neural operators, like networks, are built of layers (see [5] for background on neural networks). The input layer for a neural operator is a function $f \in \mathcal{U}$. In practice, the input layer is often a finite representation of a function, such as discretised point evaluation. The output layer is a function $v \in \mathcal{V}$. The first term in layer i of a neural operator is the integral of a kernel $K^{(i)}$ on a compact domain $\Omega_i \subset \mathbb{R}^{d_i}$ against the output of the previous layer v_i. We compose this integral with a non-linear "bias" function $b_i \in \mathbb{R}^{d_i}$ and apply an activation function σ to this expression to get input for the next layer, v_{i+1}. This process can be written as

$$v_{i+1}(x) = \sigma\left(\int_{\Omega_i} K^{(i)}(x,y)v_i(y)\mathrm{d}y + b_i(x)\right), \quad x \in \Omega_{i+1}. \tag{8}$$

The integration of the kernel against v_i is a continuous analogue of the matrix multiplication in a normal neural network and different types of kernels correspond to different types of matrices [1]. We parametrise the bias and kernel functions and optimize each layer through supervised learning, see Section 2.1. Approximating the kernel and integral operators is often computationally expensive and motivates different neural operator architectures.

2.1 Supervised Learning

Suppose we have N samples each of which is composed of input features and their corresponding output features; for example when learning the coefficient-to-solution map, see (20), one sample for a discretised PDE would contain all the tuples of coordinates, coefficients, and solutions at all grid points — that is to say a sample represents one instance of a PDE in a family of PDEs. Supervised learning aims to determine the underlying map

$\Psi^\dagger : \mathcal{U} \to \mathcal{V}$ from N samples of the solution

$$\{u_n, \Psi^\dagger(u_n)\}_{n=1}^N, \quad u_n \sim \mu, \tag{9}$$

where μ is a probability measure supported on $\mathcal{U}$ which determines the distribution of training functions u_n. The choice of μ determines what is in and out-of-distribution data. For example, a neural operator trained on continuous functions will generalise poorly to discontinuous functions. In practice we get training data from physical experiments or by using PDE solvers, which means the training data is discrete. A strength of neural operators is that even when the input data is discrete with a particular grid resolution, the neural operator can be evaluated anywhere in the domain [11].

Given data (9), the approach of supervised learning is to determine an approximation $\Psi(\cdot, \theta^*)$ to $\Psi^\dagger : \mathcal{U} \to \mathcal{V}$ from the family of parametrized solutions

$$\Psi : \mathcal{U} \times \Theta \to \mathcal{V},$$

where $\Theta \subseteq \mathbb{R}^p$ is a parameter space. The training procedure aims to find the parameter θ^* that optimizes the approximation $\Psi(\cdot; \theta^*)$ of $\Psi^\dagger$, [9]. We discuss how we choose to optimize in Section 2.2.

2.2 Training and Testing

In order to train the model Ψ we use data (9) to optimize our choice of θ, meaning we know the value of $\Psi^\dagger$ for N samples. This is analogous to the process used for neural networks. Let $\mathcal{V}'$ be a Banach space that contains the range of $\Psi^\dagger$ and $\Psi(\cdot; \theta)$. Classical and weak solutions often belong to the L^2 space so we let $\mathcal{V}' = L^2$ and we define the relative squared L^2 loss function given over the N training samples as

$$L^2\left(\{\Psi(u_i, \theta), \Psi^\dagger(u_i)\}_{i=1}^N\right) = \frac{1}{N}\sum_{i=1}^N \frac{\|\Psi(u_i, \theta) - \Psi^\dagger(u_i)\|_{L^2(\Omega)}}{\|\Psi^\dagger(u_i)\|_{L^2(\Omega)}}. \tag{10}$$

In practice, each test pair $(\Psi(u_i, \theta), \Psi^\dagger(u_i))$ is discrete, so we can evaluate this loss function using the Euclidean 2 norm.

$$L_2\left(\{\Psi(u_i, \theta), \Psi^\dagger(u_i)\}_{i=1}^N\right) = \frac{1}{N}\sum_{i=1}^N \frac{\|\Psi(u_i, \theta) - \Psi^\dagger(u_i)\|_2}{\|\Psi^\dagger(u_i)\|_2}. \tag{11}$$

With a relative error chosen, the aim of the training procedure is to minimize the empirical risk. This is the expected relative error between our true solution and predicted solution, assuming the true solution belongs to the same space as the training data. This assumption is one reason why neural operators can generalise badly to data that is different in structure to the training data.

The training procedure is typically performed using the Adam optimization algorithm [7]. Once trained, we can test our neural operator against a test set of data to see how well it performs. We discuss how we generate training and test data in Section 4.

2.3 Fourier Neural Operator

The Fourier neural operator (FNO) architecture was introduced in 2020 by Li et al [12]. The idea behind the FNO is to choose translation invariant kernels in the neural layer in (8) of the form

$$K^{(i)}(\boldsymbol{x}, \boldsymbol{y}) = k^{(i)}(\boldsymbol{x} - \boldsymbol{y}), \tag{12}$$

where the integration of the $k^{(i)}$ against $v_i(y)$ is now a convolution operation [1]. Convolution in the spacial domain can be computed via pointwise multiplication in the Fourier domain, so the integral operation is implemented as

$$\int_{\Omega_i} k^{(i)}(\boldsymbol{x} - \boldsymbol{y}) v_i \mathrm{d}y = \mathcal{F}^{-1}(\mathcal{F}(k^{(i)})\mathcal{F}(v_i))(\boldsymbol{x}), \tag{13}$$

where $\mathcal{F}$ is the Fourier transform and $\mathcal{F}^{-1}$ the inverse Fourier transform, defined for $f : \Omega_i \to \mathbb{R}^{d_c}$ by

$$(\mathcal{F}f)_j(\boldsymbol{k}) = \int_{\Omega_i} f_j(\boldsymbol{x}) e^{-2i\pi \boldsymbol{x}\cdot\boldsymbol{k}} \mathrm{d}\boldsymbol{x}, \quad (\mathcal{F}^{-1}f)_j(\boldsymbol{x}) = \int_{\Omega_i} f_j(\boldsymbol{x}) e^{2i\pi \boldsymbol{x}\cdot\boldsymbol{k}} \mathrm{d}\boldsymbol{k} \tag{14}$$

for $j = 1, \ldots, d_c$. The Fourier transform takes a function from the physical space to the wave space, or frequency domain, where $\boldsymbol{k}$ is the wave number. We call (13) the Fourier layer. In order to take advantage of the Fast Fourier Transform (FFT), which reduces the numerical complexity of the transform operation, we let the kernel be a periodic function and we parametrise it using the weight vector, $\boldsymbol{\gamma}$, of Fourier coefficients. We let $\boldsymbol{\gamma}$ be truncated to a band of Fourier modes which acts as a discretisation. This $\boldsymbol{\gamma}$ is optimized during the training process. The maximum number of Fourier modes $k_{\max}$ is often chosen to be between $8 \sim 16$ as PDEs with smooth solutions experience rapid decay in Fourier coefficients. An advantage of implementing the integral transform this way is that it reduces the complexity of the operation from $O(m^2)$ operations for a standard quadrature rule to $O(m \log(m))$, where m is the number of grid point evaluations for one instance of a PDE in the training set. The Fourier layer only works with periodic boundary conditions but the bias term b_i in the neural layer (8) helps to recover non-periodic boundary conditions [12].

Now we define the FNO architecture as in [9],

$$\begin{aligned}
\Psi_{\mathrm{FNO}}(u;\theta) &= \mathcal{Q} \circ \mathcal{L}_L \circ \cdots \circ \mathcal{L}_2 \circ \mathcal{L}_1 \circ \mathcal{R}(u;\theta), \quad \forall u \in \mathcal{U}, \\
\mathcal{L}_l(v)(\boldsymbol{x};\theta) &= \sigma\left(W_l v(\boldsymbol{x}) + b_l + \mathcal{K}(v)(\boldsymbol{x};\boldsymbol{\gamma}_l)\right), \quad \forall l \in [1, L].
\end{aligned}$$

The input and output layers are $\mathcal{R}$ and $\mathcal{Q}$ which lift to and from a space with channel width d_c. Each layer $\mathcal{L}_l$ is known as a hidden layer and involves the pointwise composition of the affine transformation described above with the chosen activation function, mapping functions from $\mathbb{R}^{d_c}$ to $\mathbb{R}^{d_c}$. The Fourier coefficient weight parameter for layer l is given by $\boldsymbol{\gamma}_l$. Hyperparameters that need to be chosen when using FNOs are the Fourier channel width d_c, the width of the features channel that is passed into $\mathcal{R}$, the Fourier cut-off $k_{\max}$, the depth L and the number of features in and out of the neural operator.

We choose to use the FNO architecture in this paper since as discussed in [13] it is

1. effective at solving PDEs with periodic boundary conditions,
2. efficient to train as it uses the FFT,
3. known to work well for fluid problems.

3 Meet the Fokkers

[1] In its most general form the Fokker-Planck equation that describes the probability density of X_t in the SDE (2) takes the form

$$\partial_t u = \sum_{i,j=1}^{d} a_{ij} \frac{\partial^2 u}{\partial x_i \partial x_j} + b \cdot \nabla u, \tag{15}$$

where $a(\boldsymbol{x}) = \frac{1}{2}\sigma\sigma^T = I_2$ for diffusion $\sigma = \sqrt{2}I_2$ and $b(\boldsymbol{x}) : \mathbb{R}^d \to \mathbb{R}$ is the drift (or velocity) field. This represents the probability density of a stochastic process. The equation may be referred to as the forward or backward Kolmogorov equation depending on whether the sign in front of $b(\boldsymbol{x})$ is negative or positive respectively. We assume that the velocity field is divergence-free, meaning that $b \cdot \nabla u = 0$. From here on let $\boldsymbol{x} = (x, y)$. We assume that our domain Ω is periodic in y so that no mass can escape our system. When $\partial_t u$ is set to 0 we call the equation steady.

3.1 The Forward Committor Function

The forward committor function q^+, defined in (5), is the probability that a stochastic process will next go to the set B if it is at point $\boldsymbol{x} \in \Omega$. This function is the solution to the following backward Kolmogorov equation

$$\begin{cases} \Delta q^+(\boldsymbol{x}) + b(\boldsymbol{x}) \cdot \nabla q^+(\boldsymbol{x}) = 0, & \boldsymbol{x} \in \Omega \setminus (A \cup B) \\ q^+(\boldsymbol{x}) = 0, & \boldsymbol{x} \in \partial A \\ q^+(\boldsymbol{x}) = 1, & \boldsymbol{x} \in \partial B \\ q^+|_{y=0} = q^+|_{y=1}, & \text{(periodic boundaries).} \end{cases} \tag{16}$$

[1] "Meet the Fockers" is a 2004 comedy film.

The question of uniqueness and existence is difficult to answer when $\boldsymbol{b}$ represents a turbulent velocity field as turbulence lacks nice regularity conditions. If $\boldsymbol{b}$ is uniformly Lipschitz-continuous in $\bar{\Omega}$ then it is known that a classical solution exists [15]. The spectral method implemented in section 4.2.2 models turbulence by a truncated Fourier series, which provides a good approximation of the high-scale behaviour of turbulence but loses some of the finer detail and characteristics of turbulence. Following this model, we assume $\boldsymbol{b} \in L^2(\Omega)$. To the best of our knowledge there are no results on the regularity of a strictly elliptic second order PDE with Dirichlet and periodic boundary conditions when $\boldsymbol{b} \in L^2(\Omega)$. We conjecture that since the domain is Lipschitz and the backwards Kolmogorov operator is strictly elliptic, there is a weak solution to (16). Further research is needed to determine existence and uniqueness results to ensure that the class of solutions the neural operator tries to learn is well-defined. For the rest of this paper we assume there is a weak solution $u \in W^{2,2}(\Omega)$. This is the space of functions u that are square-integrable over Ω and that have first and second weak derivatives that are also square-integrable over Ω.

3.2 The Steady Probability Density Function

The steady, or stationary, probability density function $\rho(\boldsymbol{x})$ solves the following steady forward Kolmogorov equation

$$\begin{cases} \Delta\rho(\boldsymbol{x}) - b(\boldsymbol{x}) \cdot \nabla\rho(\boldsymbol{x}) = 0, & \boldsymbol{x} \in \Omega \setminus (A \cup B) \\ \nabla\rho \cdot \hat{\boldsymbol{n}} = 0, & \boldsymbol{x} \in \partial A \cup \partial B \\ \rho|_{y=0} = \rho|_{y=1}, & \text{(periodic boundaries)} \\ \int_\Omega \rho(\boldsymbol{x}) d\Omega = 1, & \text{(equality constraint).} \end{cases} \tag{17}$$

This equation has Neumann boundary conditions for $\boldsymbol{x} \in \partial A \cup \partial B$ which, along with the periodic boundary conditions, ensure that the probability mass cannot escape the system. However, this is not enough to ensure uniqueness in a 2-D or higher system so we have to impose the equality constraint that the solution integrated over the domain gives one; this is a necessary property for a probability density function. Looking carefully at this system one can spot that it admits the constant solution

$$\rho(\boldsymbol{x}) = 1/|\Omega|, \quad \forall \boldsymbol{x} \in \Omega.$$

This simplifies the probability density of reactive trajectories in (7) to

$$\rho_{\text{react}} = \frac{q^-(\boldsymbol{x})q^+(\boldsymbol{x})}{|\Omega|}. \tag{18}$$

3.3 The Backwards Committor Function

The backwards committor function $q^-(\boldsymbol{x})$ is the probability that a stochastic process has just come from set A given it is at a point $\boldsymbol{x} \in \Omega$. Another interpretation of the backwards committor function is as the forwards committor function of the reversed time diffusion process. It is a consequence of stochastic theory, [15, p. 12], that the generator of the backwards committor function with diffusion $\sigma = \sqrt{2}I_2$ is given by

$$\bar{L}q^- := \Delta q^- + b^R \cdot \nabla q^-,$$

where

$$b^R(\boldsymbol{x}) = -b(\boldsymbol{x}) + \frac{2}{\rho(x)}\nabla \cdot (a(\boldsymbol{x})\rho(\boldsymbol{x})).$$

This simplifies due to us showing $\rho = 1/|\Omega|$ in Section 3.2. Therefore the backwards committor function is the solution to the forward Kolmogorov equation,

$$\begin{cases} \Delta q^-(\boldsymbol{x}) - b(\boldsymbol{x}) \cdot \nabla q^-(\boldsymbol{x}) = 0, & \boldsymbol{x} \in \Omega \setminus (A \cup B) \\ q^-(\boldsymbol{x}) = 1, & \boldsymbol{x} \in \partial A \\ q^-(\boldsymbol{x}) = 0, & \boldsymbol{x} \in \partial B \\ q^+|_{y=0} = q^+|_{y=1}, & \text{(periodic boundaries).} \end{cases} \tag{19}$$

4 Generating Training Data

The aim is to train the neural operator to learn the "coefficient and domain to solution" map

$$(b(x,y), f^{-1}(x,y)) \to \rho_{\text{react}}(x,y), \tag{20}$$

where $b : \Omega \subset \mathbb{R}^2 \to \mathbb{R}^2$ is a turbulent velocity field, $f^{-1} : [0,1]^2 \to \mathbb{R}$ contains information about the shape of the boundaries, and $\rho_{\text{react}} : \Omega \subset \mathbb{R}^2 \to [0,1]$ is a probability density on the space. The inputs will actually be given on a discretisation of the space Ω and the output will work as a black box function on any point we input.

To generate training data we use a finite difference PDE solver on a discretisation of equations (16) and (19). First we discuss how the domain Ω is formulated and generated in Section 4.1. Then we discuss turbulence and how to generate synthetic turbulence in a domain in Section 4.2. Finally, in Section 4.3 we formulate finite difference PDE solvers for equations (16) and (19) on the generated domains with turbulent drift coefficient b.

4.1 Generating Domains

The domain of the problem is a neighbourhood of Ω of $[0,1]^2$, representing streams of particle movement at the left and right boundary as in Figure 1.

The aim is to generate boundaries for the vector field with certain properties. Let

$$r_\phi(y) = (\phi(y), y) \quad \text{and} \quad r_\psi(y) = (\psi(y), y), \tag{21}$$

where $\phi, \psi : \mathbb{R} \to \mathbb{R}$ are the left boundary and right boundary as in Figure 2. In order to model physical systems we impose differentiability on ϕ and ψ. These are regular parametrisations as $r'_\phi(y), r'_\psi(y) \neq \mathbf{0}$ for $y \in [0, 1]$. To allow for periodic boundary conditions at the top and bottom of the space Ω we want $\phi(0) = \phi(1) = 0$ and $\psi(0) = \psi(1) = 1$. Following these conditions it makes sense to generate these curves as random trigonometric polynomials of sine functions. Algorithm 1 summarizes the process.

Algorithm 1 Generating ϕ and ψ in (21)

1: The integers N_1 and N_2 are chosen from the $[1, N_{\max}]$ with uniform probability. These are the orders of the trigonometric polynomials ϕ and ψ.

2: Generate N_1 and N_2 random coefficients, $\{c_{1,i}\}_{i=1}^{N_1}$ and $\{c_{2,i}\}_{i=1}^{N_2}$, between $[-1, 1]$ for ϕ and ψ, respectively.

3: Define

$$\phi_{\text{temp}}(y) = \sum_{k=1}^{N_1} c_{1,k} \sin(k\pi y) \quad \text{and} \quad \psi_{\text{temp}}(y) = \sum_{k=1}^{N_2} c_{2,k} \sin(k\pi y).$$

4: Calculate the normalizing constants:

$$\phi_{\text{norm}} = 3 \cdot \max_{y \in [0,1]} \phi_{\text{temp}}(y) + \text{mean}_{y \in [0,1]} \phi_{\text{temp}}(y),$$

and similarly for ψ_{norm}.

5: Define the normalized functions:

$$\phi(y) = \frac{\phi_{\text{temp}}(y)}{\phi_{\text{norm}}} \quad \text{and} \quad \psi(y) = \frac{\psi_{\text{temp}}(y)}{\psi_{\text{norm}}} + 1.$$

6: If $|\phi(y) - \psi(y)| < \epsilon$ for some $y \in [0, 1]$ and some chosen tolerance ϵ then discard and generate again with new random numbers.

The "normalizing step" 5 does not strictly speaking normalize the curve but reduces the amplitudes of the sine components in a sensible way so that both curves lie in a 0.5-neighbourhood of the domain Ω. For a physical system we need the boundaries r_ψ and r_ϕ in (21) to not overlap and for there to be some minimum distance, ϵ, between the two functions $\phi(y)$ and $\psi(y)$; a new pair of curves can be generated if these conditions do not hold. We arbitrarily choose $N_{\max}$, the maximum order of the trigonometric polynomials, to be 8 so that the curves are not too "bumpy".

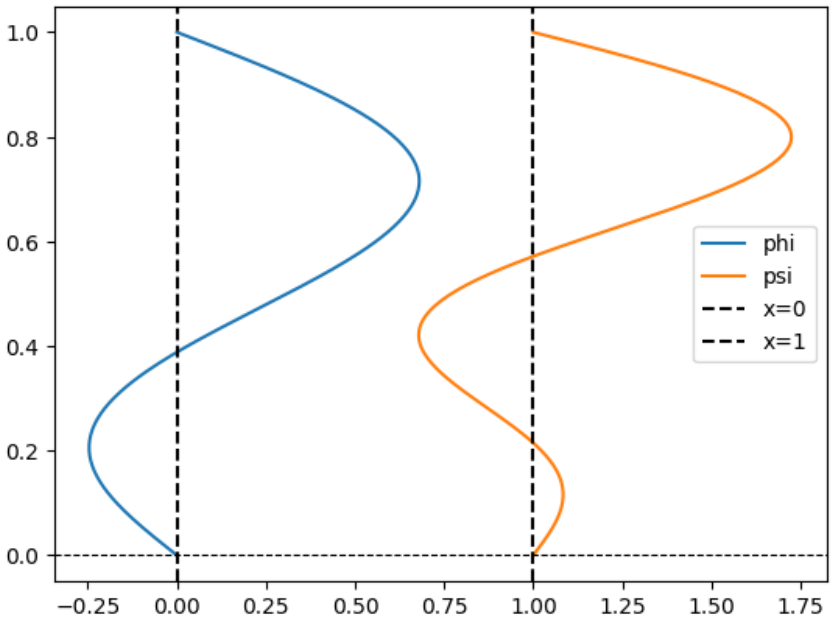

Figure 2: Example of boundary curves parametrised by $\phi(y)$ and $\psi(y)$ using Algorithm 1.

4.2 Generating a Turbulent Velocity Field

In order to generate velocity fields, $b(\boldsymbol{x})$, that represent turbulence it is important to understand what properties characterise turbulence. As turbulence theory is not the main focus of this paper, we summarize key details to motivate Algorithm 2, informed by [18]. In this discussion we use the standard notation $\boldsymbol{U}(\boldsymbol{x}, t)$ to represent a time-dependent velocity field.

4.2.1 Turbulence Theory

Turbulence can be modelled as a velocity field that varies significantly and irregularly in space and time, known as a random field [19]. When solving the steady Kolmogorov equations we will use a snapshot of turbulence in time. However in this section we discuss turbulence over time. Turbulent flows are characterised by being rotational and having a mixture of large and small scale motions. One of the key ideas of turbulence is the energy cascade, where kinetic energy enters the turbulence at the largest scale of motions and is transferred to smaller and smaller scales. A famous quote by Lewis Fry Richardson summarizes this well, [20][p. 66]:

> "Big whirls have little whirls that feed on their velocity,
> and little whirls have lesser whirls and so on to viscosity."

We call these circular whirls eddies. The velocity field, $\boldsymbol{U}(\boldsymbol{x}, t)$, can be decomposed as

$$\boldsymbol{U}(\boldsymbol{x}, t) = \boldsymbol{u}(\boldsymbol{x}, t) + \langle \boldsymbol{U}(\boldsymbol{x}, t) \rangle \tag{22}$$

where $\boldsymbol{u}(\boldsymbol{x},t)$ represents the fluctuation and $\langle \boldsymbol{U}(\boldsymbol{x},t)\rangle$ represents the mean, or expectation, of the velocity field. This is the Reynolds decomposition. For most of the turbulence theory the fluctuation,$\boldsymbol{u}$, is used in place of $\boldsymbol{U}$ as it is the important part statistically.

Definition 4.1 (Statistically homogeneous random fields)**.** A random field is *homogeneous* if all statistics are invariant under a shift in position.

Homogeneous turbulence is less restrictive and requires only that the fluctuation velocity field, $\boldsymbol{u}(\boldsymbol{x},t)$ is statistically homogeneous.

A key idea in turbulence modelling is to compare statistics about turbulence in physical space to the energy spectrum in Fourier space. To motivate the use of Fourier transforms we introduce the two-point one-time auto-covariance

$$R_{ij}(\boldsymbol{r},\boldsymbol{x},t) := \langle u_i(\boldsymbol{x},t)u_j(\boldsymbol{x}+\boldsymbol{r},t)\rangle\,,$$

where $\boldsymbol{x},\boldsymbol{r}\in\mathbb{R}^2$. Then assume that we have homogeneous turbulence and so that we can drop dependence on $\boldsymbol{x}$. We can then define the velocity spectrum tensor as

$$\Phi_{ij}(\boldsymbol{k},t) := \frac{1}{(2\pi)^2}\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{i\boldsymbol{k}\cdot\boldsymbol{r}}R_{ij}\,\mathrm{d}\boldsymbol{r},$$

where $\boldsymbol{k}\in\mathbb{R}^2$ represents spacial frequency of a Fourier mode, $e^{i\boldsymbol{k}\cdot\boldsymbol{r}}$. The two-point one-time auto-covariance,R_{ij}, and the velocity spectrum tensor, Φ_{ij}, form a Fourier-transform pair since one can write

$$R_{ij}(\boldsymbol{r},t) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty} e^{i\boldsymbol{k}\cdot\boldsymbol{r}}\Phi_{ij}(\boldsymbol{k},t)\,\mathrm{d}\boldsymbol{k}.$$

We can think of $\Phi_{ij}(\boldsymbol{k},t)$ as the contribution of the covariance $\langle \boldsymbol{u}_i\boldsymbol{u}_j\rangle$ of velocity modes with wave number $\boldsymbol{k}$. With the velocity tensor we can define the energy spectrum

$$E(k,t) := \int_{-\infty}^{\infty}\int_{-\infty}^{\infty}\frac{1}{2}\Phi_{ij}(\boldsymbol{k},t)\delta(|\boldsymbol{k}|-k)\,\mathrm{d}\boldsymbol{k}, \tag{23}$$

where δ is the Dirac delta function which picks out wave numbers $\boldsymbol{k}$ with magnitude k. Turbulent motions with physical length scale l have a wave number with size $k=2\pi/l$. When the turbulence is homogeneous and statistically stationary in time we only need to consider the trace of the velocity spectrum tensor and can write the energy spectrum function without any directional or temporal information as

$$E(k) = \int_{-\infty}^{\infty}\int_{-\infty}^{\infty}\frac{1}{2}\Phi_{ii}(\boldsymbol{k})\delta(|\boldsymbol{k}|-k)\,\mathrm{d}\boldsymbol{k}.$$

Understanding the energy spectrum in a turbulent system helps with incorporating the key features of turbulence in a spectral model. First we define an important parameter

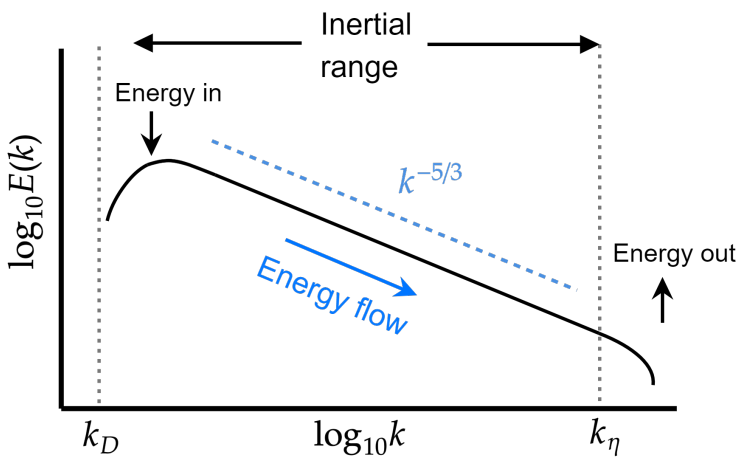

Figure 3: The energy cascade with respect to the wave number magnitude, k, for turbulence with high Reynolds numbers. Reproduced with license from Pstasiak, CC BY-SA 4.0 via Wikimedia Commons.

in characterizing turbulence. The Reynolds number, Re, can be derived from the Fokker-Planck equation (3) as the only free parameter when performing dimensional analysis. It is defined as

$$\mathrm{Re} := \mathcal{U}\mathcal{L}/\nu, \tag{24}$$

where $\mathcal{U}$ is the characteristic velocity scale, $\mathcal{L}$ is the characteristic length scale, and ν is the characteristic viscosity. We can think of this number as the ratio of inertial and viscous terms and it can characterise the dynamics we see in the turbulence. We treat $\mathcal{L}$ as the largest size of an eddy that our turbulent system admits.

Kolmogorov published an impactful paper in 1964 that proposed the idea of an energy cascade in turbulence [8]. Figure 3 shows the energy cascade for turbulence with a high Reynolds number with respect to the magnitude, k, of the wave number, $\boldsymbol{k}$. We call the lowest range of values in this diagram the *energy-containing range*, which corresponds to the wave numbers for the largest eddies in the turbulence that gain kinetic energy. Next, the *inertial subrange* represents the transfer of energy to smaller scales (smaller eddies) until the *dissipation range* is reached and the energy leaves the system. A key conclusion from Kolmogorov's second hypothesis is that in the inertial range the energy spectrum is

$$E(k) = C\varepsilon^{2/3}k^{-5/3}, \tag{25}$$

where C is a universal constant and ε is the dissipation rate. This is known as Kolmogorov's $k^{-5/3}$ law. With this theory we can approximate turbulence with spectral methods.

4.2.2 Turbulence Algorithm

We assume that our turbulent velocity field is homogeneous, periodic, and divergence-free. The periodic assumption works well with the y-periodicity in the domains for our Fokker-Planck equations. We write the Fourier series of the turbulent velocity field as

$$\boldsymbol{u}(\boldsymbol{x}, t) = \sum_{\boldsymbol{k}} \hat{\boldsymbol{u}}(\boldsymbol{k}, t) e^{i\boldsymbol{k}\cdot\boldsymbol{x}}, \tag{26}$$

where the Fourier coefficients for a given $\boldsymbol{k}$ and t are given by the Fourier transform of $\boldsymbol{u}(\boldsymbol{x}, t)$. The time component is important for understanding turbulence but since this paper studies steady Fokker-Planck equations we can fix the time from here on and generate a snapshot of an instance of turbulence.

The idea of Algorithm 2 is that we project our discretisation of the physical space into Fourier space. There we select random Fourier modes, $e^{i\boldsymbol{k}\cdot\boldsymbol{x}}$, within the inertial range of the spectrum and we generate random Fourier coefficients associated with these modes. Then we apply Kolmogorov's $k^{-5/3}$ law to scale the Fourier coefficients with wave number k by $k^{-5/3}$ so that the energy spectrum of the inertial range in (25) is satisfied. We project the velocity field to divergence-free space. Finally we take the inverse Fourier transform of our series.

In order to determine the inertial range of the spectrum, we discretise our domain Ω with resolution $N \times N$; this spacial approximation carries through into the Fourier domain. As previously stated, turbulent motions with physical length scale l have a wave number with size $k = 2\pi/l$. Given our characteristic length scale $\mathcal{L}$ this means

$$k_{\text{min}} = \frac{2\pi}{\mathcal{L}}, \quad k_{\text{max}} = \frac{\pi N}{\mathcal{L}}. \tag{27}$$

Hence the smallest wave number represents the largest motion in the system and the largest wave number represents the smallest motion. In the literature k_{max} is also called the Nyquist limit.

The divergence-free condition for our velocity field $\boldsymbol{u} = (u, v)$ can be projected to Fourier space by taking the Fourier transform of $\partial_x u$ and of $\partial_y v$ to get

$$ik_x\hat{u} + ik_y\hat{v} = 0, \tag{28}$$

where $\boldsymbol{k} = (k_x, k_y)$. As a result we can project the velocity field in Fourier space to a divergence-free space by computing

$$\hat{u}' = \hat{u} - ik_x\hat{v}, \quad \hat{v}' = \hat{v} - ik_y\hat{v}. \tag{29}$$

The following algorithm was generated using example code from Tobias Grafke and theory

from Pope [18].

Algorithm 2 Generating a turbulent vector field

1: Input: the Reynolds number, Re, the characteristic length, $\mathcal{L}$, and the characteristic viscosity ν. Also pick a resolution for the discretisation of the domain.
2: Centre the domain at the Cartesian origin, and apply the fast Fourier transform to define the grid in Fourier space. Each grid point is associated to a wave number $\boldsymbol{k}$.
3: For Fourier modes with wave number size k in the range $[k_{\min}, k_{\max}]$ draw a random number to be the associated Fourier coefficient, then scale by $k^{-5/3}$.
4: Project to divergence free space.
5: Transform the velocity field to physical space with an inverse fast Fourier transform, and undo the centring of the domain.
6: Rescale the velocity field to have average speed $\mathcal{U} = \nu \mathrm{Re}/\mathcal{L}$ by equation (24).

Normally this pseudo-spectral approach is used to generate an initial turbulent vector field and then a Navier-Stokes simulation is run for some time to make the turbulence more physical; this is because we are drawing velocity fields that have the correct energy spectrum but lack structures prevalent in fluid flow, like vortices. Instead we assume Algorithm 2 models turbulence well enough for application to operator learning in this paper. The use of statistical "synthetic" fields for reasoning about transport is common in the literature: the Kraichnan model for advection-diffusion equations assumes that the vector field is homogenous, isotropic (invariant under rotations and reflections), and Gaussian [10]. The approach in Algorithm 2 requires a rectangular domain for generating the Fourier modes and since our transformation of the domain Ω to a square does not preserve the divergence-free property it is important that this turbulent vector field is generated in the original domain Ω. As a result we generate turbulence on a rectangular ϵ-neighbourhood of Ω, meaning we extend outwards horizontally so that Ω is a subset of the rectangle, and then restrict the turbulence to this domain and assume that this is still a useful model of turbulence.

To generate the turbulent velocity fields we pick $\mathcal{L} = 0.2, \nu = 1$ and $\mathrm{Re} = 10$: by (24) this gives a characteristic velocity of $\mathcal{U} = 50$. These are chosen somewhat arbitrarily but ensure that we can fully resolve the turbulence and that our implementation is numerically stable. In practice ν is of the order $10^{-4} \sim 10^{-6}$ and so in this case it would mean Re is between $10^5 \sim 10^7$.

4.3 Finite Difference Solvers

In order to generate training data for the neural operator, we develop numerical solvers for the committor functions in (16) and (19). Recall that by equation (18) we only need to solve the PDEs for the forward and backward committor functions. We choose to solve these PDEs using finite difference schemes, which numerically approximate the solution on a discretisation of the domain. The main challenge is dealing with the curved domains

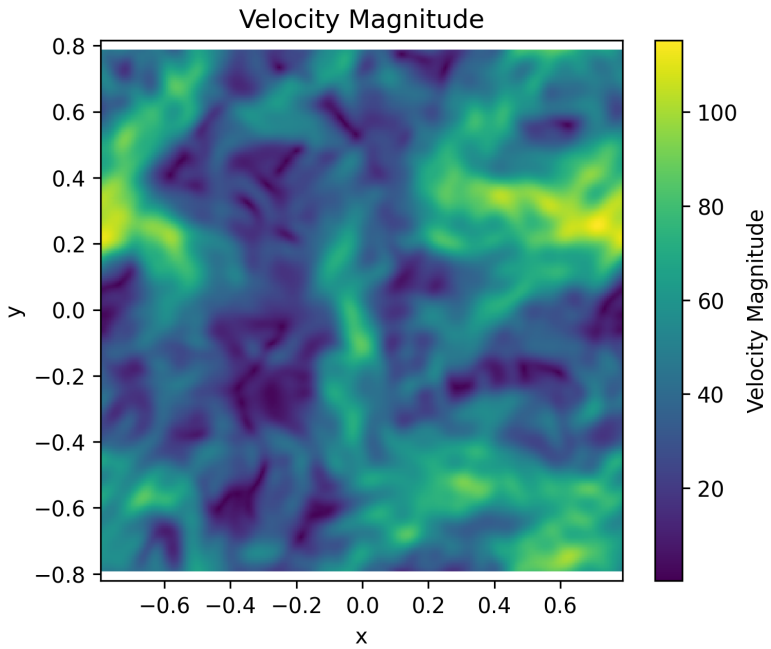

Figure 4: Example generated turbulent velocity field. Note no indication of direction of flow is given.

for Ω set out in section 4.1.

We decide to project from the domain Ω into a square domain $[0,1]^2 \subset \mathbb{R}^2$ and solve there before projecting the solution back to the original space. The reason this works is because the top and bottom of Ω, in having periodic boundary conditions, are parallel lines of the same length so a simple bijection between Ω and $[0,1]^2$ can be established. We define the coordinate transforms

$$(x,y) \in \Omega \to (f(x,y),y) \in [0,1]^2, \quad \text{where } f(x,y) = \frac{x-\phi(y)}{\psi(y)-\phi(y)}. \tag{30}$$

We also define the inverse of f

$$f^{-1}\left(f(x,y),y\right) = \phi(y) + f(x,y)\left(\psi(y)-\phi(y)\right) = x, \tag{31}$$

which is used as an input feature in (20). In this case, f^{-1} should be thought of as giving the x coordinate for the original domain Ω.

By passing this coordinate transform to the neural operator as a feature during training, the hope is that the neural operator can predict solutions for all domains with boundaries defined as in Section 4.1. The downside for developing solvers is that this coordinate transform affects the PDEs we consider. Theorem 4.1 demonstrates a new result on how the coordinate transform (30) affects the steady advection-diffusion equation (32).

Theorem 4.1 (Forward committor operator under coordinate transform)**.** *If we let* $q^+(x,y) =$

$u(f(x,y),y)$ and $b(\boldsymbol{x}) = (b_1(\boldsymbol{x}), b_2(\boldsymbol{x}))^T$ then the following expressions are equivalent:

$$\Delta q^+(x,y) + b.\nabla q^+(x,y) = 0, \quad \forall (x,y) \in \Omega \tag{32}$$

and

$$\Delta u + \left(\frac{\partial f}{\partial y}\right)^2 \partial_{\hat{x}\hat{x}} u + 2\frac{\partial f}{\partial y}\partial_{\hat{x}y} u + \left(\partial_{yy} f + b_1 + b_2 \frac{\partial f}{\partial y}\right)\partial_{\hat{x}} u + \partial_y u = 0, \quad \forall\, (f(x,y),y) \in [0,1]^2 \tag{33}$$

where $\hat{x} = f(x,y)$ and we take Δu to mean $\partial_{\hat{x}\hat{x}} u + \partial_{yy} u$.

Proof. The chain rule for multivariable functions in $\boldsymbol{x} = (x_1, \ldots, x_n)$ is given by

$$\frac{d}{dx_i} f\left(g_1(\boldsymbol{x}), g_2(\boldsymbol{x}), \ldots, g_n(\boldsymbol{x})\right) = \sum_{i=1}^{n} \left(\frac{d}{dx_i} g_i(\boldsymbol{x})\right) D_i f(g_1(\boldsymbol{x}), g_2(\boldsymbol{x}), \ldots, g_n(\boldsymbol{x})), \tag{34}$$

where $D_i f$ is the partial derivative of f with respect to the ith argument. Using this, and suppressing notation from $u(f(x,y),y)$ to $u(f,y)$ we find

$$\nabla q^+(x,y) = \nabla u(f,y) = \begin{pmatrix} \frac{\partial f(x,y)}{\partial x} D_1 u(f,y) \\ \frac{\partial f(x,y)}{\partial y} D_1 u(f,y) + D_2 u(f,y) \end{pmatrix}$$

and

$$\begin{aligned} \Delta q^+(x,y) &= \nabla \cdot \nabla u(f,y) \\ &= \frac{\partial^2 f(x,y)}{\partial x^2} D_1 u(f,y) + \left(\frac{\partial f(x,y)}{\partial x}\right)^2 D_1^2 u(f,y) \\ &\frac{\partial^2 f(x,y)}{\partial y^2} D_1 u(f,y) + \frac{\partial f(x,y)}{\partial y}\left(\frac{\partial f(x,y)}{\partial y} D_1^2 u(f,y) + D_2 D_1 u(f,y)\right) \\ &+ \frac{\partial f(x,y)}{\partial y} D_1 D_2 u(f,y) + D_2^2 u(f,y). \end{aligned}$$

We assume (see Section 3.1) that q^+ is smooth enough to switch the order of the mixed partial derivatives. It is clear from equation (30) that $\frac{\partial f(x,y)}{\partial x} = 1$ and $\frac{\partial^2 f(x,y)}{\partial x^2} = 0$. Applying these expressions and substituting them into equation (32) we get the desired result. □

Now we introduce some notation for finite differences. Let $h = 1/(N+1)$ be the grid step-size. Let $\Omega' = [0,1]^2$ and define

$$\Omega'_h := \{(ih, jh) : i, j \in [0, N+1]\}.$$

This is a uniform discretisation of the domain where N represents the number of interior points in one grid row or column. Thus Ω'_h contains $(N+2) \times (N+2)$ points. Let

$u_h : \Omega'_h \to \mathbb{R}$ be the restriction of u to Ω'_h and let $u_{i,j} = u_h(\frac{i}{N+1}, \frac{j}{N+1})$. We use the following differences to approximate the derivatives in equation (33).

Definition 4.2 (Finite differences in 2-D)**.** Let h be a given step size. For a continuous function $u : \mathbb{R}^2 \to \mathbb{R}$ and a point $\boldsymbol{x} \in \Omega'_h$ we define

- The forward difference
$$D_x^+ u_{i,j} := \frac{u_{i+1,j} - u_{i,j}}{h},$$
- the backward difference
$$D_x^- u_{i.j} := \frac{u_{i,j} - u_{i-1,j}}{h},$$
- the second difference
$$D_x^2 u_{i,j} := \frac{u_{i+1,j} - 2u_{i,j} + u_{i-1,j}}{h^2}.$$

The differences are defined analogously in y. Finally the mixed difference is given by

$$D_{xy} u_{i,j} = \frac{u_{i+1,j+1} - u_{i+1,j-1} - u_{i-1,j+1} + u_{i-1,j-1}}{4h^2}$$

From first principles it is easy to show that the finite differences tend to the intended differential operator as $h \to 0$.

Before going into detail about the finite difference solvers we discuss upwinding. Since the randomly generated turbulence in section 4.2.2 varies significantly and irregularly in space we can run into stability issues when using just forwards or backwards differences for the advection term. In an upwinding scheme you choose the difference based on the sign of the advection term. Writing $b \cdot \nabla u = b_x \partial_x u + b_y \partial_y u$ we approximate the first term by

$$[b_x \partial_x u]_h = \begin{cases} b_x D_x^- u, & b_x \geq 0, \\ b_x D_x^+ u, & b_x < 0. \end{cases} \tag{35}$$

The idea is that, considering the x-direction, if $b_x \geq 0$ then a particle transported by advection is moving to the right and so we want our difference to be looking "upstream" and using the information (downstream) that we already have.

4.3.1 Forward Committor Solver

The transformed PDE system that we solve looks like

$$\begin{cases} \Delta u + \left(\frac{\partial f}{\partial y}\right)^2 \partial_{\hat{x}\hat{x}} u + 2\frac{\partial f}{\partial y}\partial_{\hat{x}y} u + \left(\partial_{yy} f + b_1 + b_2 \frac{\partial f}{\partial y}\right)\partial_{\hat{x}} u + \partial_y u = 0, & \boldsymbol{x} \in (0,1) \times [0,1] \\ u(\boldsymbol{x}) = 0, & x = 0 \\ u(\boldsymbol{x}) = 1, & x = 1 \\ u|_{y=0} = u|_{y=1}, & \text{(periodic boundaries).} \end{cases} \tag{36}$$

We assume that the domain shape has been generated as in Section 4.1, so we have a function $f(x, y)$ and its derivates which we can evaluate at the grid points. To solve the discretised forward committor equation we solve the linear system

$$Au = B, \quad A \in \mathbb{R}^{N(N+2)\times N(N+2)}, \quad u \in \mathbb{R}^{N(N+2)}, \quad B \in \mathbb{R}^{N(N+2)},$$

where u has ordering

$$u = \left(u_{1,0}, u_{1,1}, u_{1,2}, \ldots, u_{1,N+1}, u_{2,0}, u_{2,1}, \ldots, u_{N,N+1}\right)^T.$$

We apply upwinding which makes the scheme difficult to visualise. For each $u_{i,j}$ with $i \in [1, N]$ and $j \in [0, N+1]$ we apply the differences to equation (33) and fill row $(N+2)(i-1)+j$ of the matrix A with the coefficients corresponding to each term of u. To account for periodicity we apply modular arithmetic so that the terms at $j = 1$ use terms at $j = N+1$ for differences in y, and vice versa for $j = N$ and $j = 0$. Finally, the Dirichlet boundary conditions are included in the vector B: whenever the discretisation in x at $i = 1$ or $i = N$ used the terms $u_{0,j}$ or $u_{N+1,j}$, we replace with the boundary conditions and take across to B. This is why A is not a square matrix.

Since this is a sparse system, when we implement the solver in Python we use the sparse solver "spsolve" from Scipy's sparse linear algebra library.

4.3.2 Backward Committor Solver

Following the method of Theorem 4.1 we find that we need to solve

$$\begin{cases} \Delta u + \left(\frac{\partial f}{\partial y}\right)^2 \partial_{\hat{x}\hat{x}} u + 2\frac{\partial f}{\partial y}\partial_{\hat{x}y} u + \left(\partial_{yy} f - b_1 - b_2 \frac{\partial f}{\partial y}\right)\partial_{\hat{x}} u + \partial_y u = 0, & \boldsymbol{x} \in (0,1) \times [0,1] \\ u(\boldsymbol{x}) = 1, & \boldsymbol{x} = 0 \\ u(\boldsymbol{x}) = 0, & \boldsymbol{x} = 1 \\ u|_{y=0} = u|_{y=1}, & \text{(periodic boundaries).} \end{cases} \tag{37}$$

We solve this using the same method as for the forward committor solver. We implement

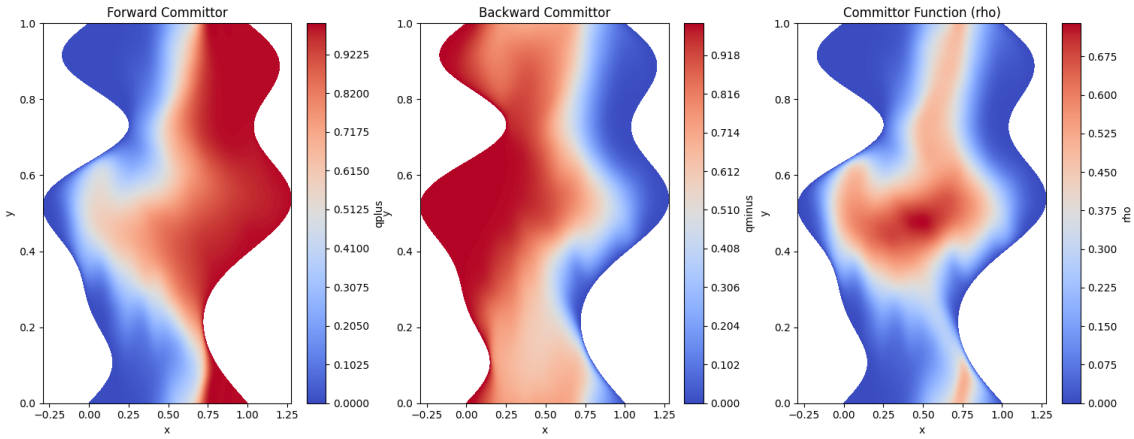

Figure 5: Example solution to forward and backward committor functions using finite difference solvers, and resulting value of ρ_{react}.

our Kolmogorov solvers for the forward and backward committor functions in Python and use the package "autograd" to accurately compute the derivatives of $f(x, y)$ at each point.

We can get our discrete approximation to ρ_{react} by multiplying the solution to the forward and backwards committor solvers. Figure 5 shows one such solution: we have generated a domain and turbulent field as detailed in sections 4.1 and 4.2.2. The more red a point is, the higher the probability density is at that point. Hence the red indicates the transition paths. Since ρ_{react} is the product of the two committor function we expect ρ_{react} to have some aggregation of probability density in the middle of the domain. We see a clear path emerge out from the centre to take the stochastic process to set B with highest probability.

5 Training the Neural Operator

We now set up a number of experiments to compare the performance of the FNO across different discretisation resolutions and training sizes. First we see results for when Ω is just a square $[0, 1]^2$, so no coordinate transform. Then we see how the FNO performs on the transformed domain problem. For easy distinction we will call this the *curved domain*. These experiments are run on an Apple M1 Pro chip with 16GB of memory.

We generate training data by running the finite difference solvers on a 311×311 grid and then sub-sample from the inputs and solutions to generate training data at different grid resolutions (16×16, 32×32, and 63×63). We choose the resolution 311 as with it the solvers are able to fully resolve the PDE system and it is easy to subsample 32 points from 311 (on $[0, 1]$ a step-size of $1/31$ divides the interval into 32 points).

We use the *gaussian error linear unit* (GELU) as our activation function. There is experimental evidence that it performs well at capturing detail from the Fourier layers in FNOs

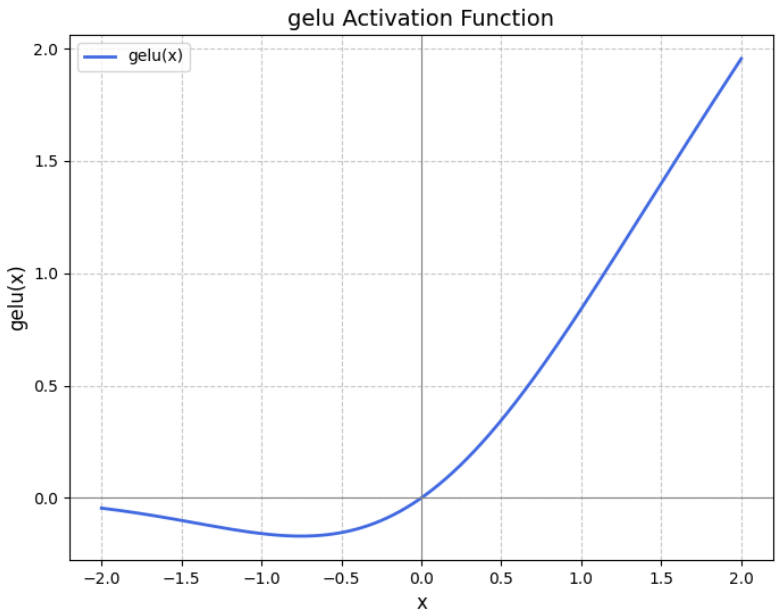

Figure 6: The gelu activation function (38).

[2]. This is a smooth non-linear function of the form

$$\mathrm{GELU}(x) = x \cdot \Theta(x), \tag{38}$$

where $\Theta(x)$ is the cumulative distribution function of the standard normal distribution. Figure 6 illustrates the GELU activation function. Unlike the *rectified linear unit* (ReLU), which zeroes out negative values, GELU applies a smooth decay to negative inputs, allowing for more gradual gradient updates during back propagation (part of the optimization procedure).

The main metric that we use to analyse the neural operator's performance is the relative L_2 loss error calculated using the loss function (11). Since the space of solutions of PDEs is an infinite dimensional vector space, norms are not equivalent. The relative L_2 loss is commonly chosen in the literature on FNOs because training has been found to successfully reduce this error and because it leads to the operator acting more smoothly over the input space [1, pg. 28]. Figure 7 shows the reduction in training loss when training an FNO over 100 epochs (how many times the optimization algorithm passes over the training set) for 1000 PDE instances on the square domain, using the relative L_2 loss function.

For all of these experiments we employ a learning rate of 0.001 - this is the step-size that the gradient descent algorithm in the optimization procedure makes in the parameter space [5]. This is a standard choice in the literature [1].

Optimizing hyperparameters is a "dark art" and often relies on experience and intuition.

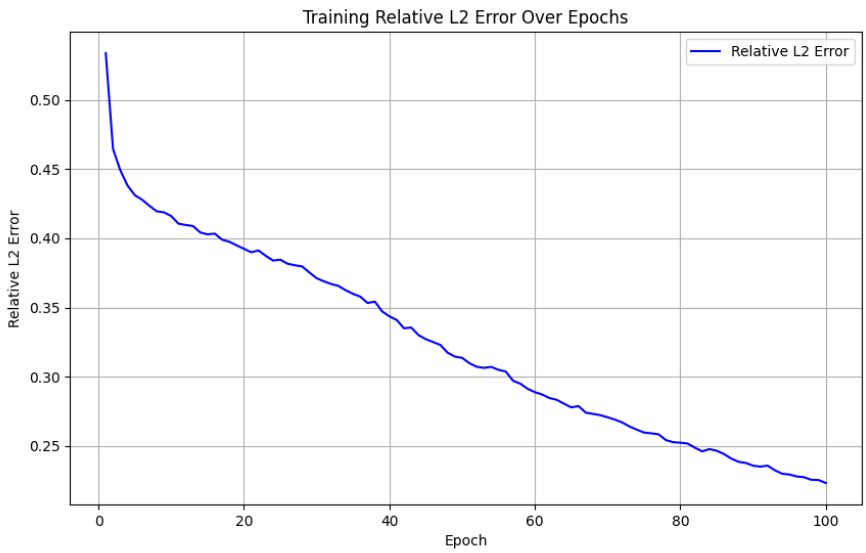

Figure 7: Training loss for 1000 PDE instances at resolution 32×32 on square domain using the relative L_2 loss error (10).

Table 1: Relative L_2 test error (11) on a square domain for different data resolution grids: $s \times s$ for the training data and $s' \times s'$ for the testing data.

Resolutions	s' = 16	s' = 32	s' =63
$s = 16$	**0.424967**	0.444524	0.447990
$s = 32$	0.832189	**0.411058**	0.429767
$s = 63$	0.529199	0.431389	**0.409939**

Hence none of these results are fully optimized. Sometimes a Python package "DeepHyper" is used to optimize hyperparameters with advanced search algorithms but this is very computationally intensive and requires the use of high performance computers [21].

5.1 Square Domain Results

We let $\Omega = [0, 1]^2$. Our input features are $\{x, y, b_1, b_2\}$, where x, y are coordinates and b_1, b_2 are the components of b evaluated at these coordinates. The output feature is $\{\rho_{\text{react}}(x, y)\}$.

First we test the generalization of the FNO across resolutions by training using data with resolution $s \times s$ and testing on data of resolution $s' \times s'$ as done in [9]. We train using 1000 PDE instances for 100 epochs with batch size 32 (how the data is grouped together for the optimization algorithm). We fix the hyperparameters as, $k_{\max} = 12$, Fourier channel width $d_c = 128$, input channel width 256; this is not the best choice of hyperparameters for each resolution but is the same for each to make results comparable. The test sets have 200 PDE instances of resolution $s' \times s'$.

Table 1 shows that the FNO maintains a similar test error when predicting at finer res-

olutions than it was trained (right of the bold diagonal), but the test error grows when testing against smaller resolutions. The property that allows neural operators to predict well at finer resolutions than they have been trained is called zero-shot super-resolution; this represents the fact that neural operators learn a class of solutions that is invariant to discretisation. The FNO trained at 63×63 resolution is notably better at predictions on coarser resolutions than the FNO trained at 32×32 resolution: this result occurs in the literature [11].

By reducing the number of epochs to 65 we are able to train an FNO of resolution 32×32 with relative test error 0.323, against the test dataset of resolution 32×32. Figure 8 shows the predictions this operator makes for 3 PDE instances. This FNO is able to capture the rough shapes at this resolution but loses some of the finer detail. Next we train an FNO on a 63×63 resolution version of the same dataset for 70 epochs, with all other hyperparameters the same. This gives a relative test error of 0.335. Figure 9 shows the predictions this FNO makes for the same 3 PDE instances as the last FNO. The shape of the middle instance is clearer but the solution within all three instances is quite rough compared to the ground truth. The FNO is predicting the shapes of the transition paths well with these hyperparameters but is struggling to concentrate the probability density smoothly within these shapes.

5.2 Curved Domain Results

Now Ω is some neighbourhood of $[0,1]^2$. Our input features are $\{x, y, b_1, b_2, f^{-1}\}$, where x, y are coordinates of the box $[0,1]^2$, b_1, b_2 are the components of b evaluated at the point $(f^{-1}(x,y), y) \in \Omega$ and f^{-1} is evaluated at (x,y). The output feature is again $\{\rho_{\text{react}}(x,y)\}$.

We start again by testing how the FNO trained at resolution $s \times s$ performs against test data of resolution $s' \times s'$. Again we train 1000 PDE instances for 100 epochs but now the batch size is 64, $k_{\max} = 16$, $d_c = 256$, and input channel width is 256. Again this is not an optimized hyperparameter choice but it is chosen to reflect the increasing complexity of the data with the additional feature f^{-1}. Table 2 shows the results of tests again 200 PDE instances of resolution $s' \times s'$. We see the zero-shot super-resolution property again. However the losses are higher than in Table 1 reflecting the increased complexity of this system.

Next we generate an FNO with 5000 32×32 resolution PDE instances; the only change in hyperparameters is that the batch size is 32 and the number of epochs 80. This FNO achieves a relative test error of 0.466 on the test set of 200 PDE instances. This was the lowest test error we were able to achieve which suggests the parameter space is more difficult to optimize over with the inclusion of the domain input feature, f^{-1}. Figure 10 shows the predictions this FNO made for 3 PDEs against the ground truth. The FNO appears worse at capturing the shapes of the transition paths, struggling notably with the zigzag shapes in the second and third PDEs.

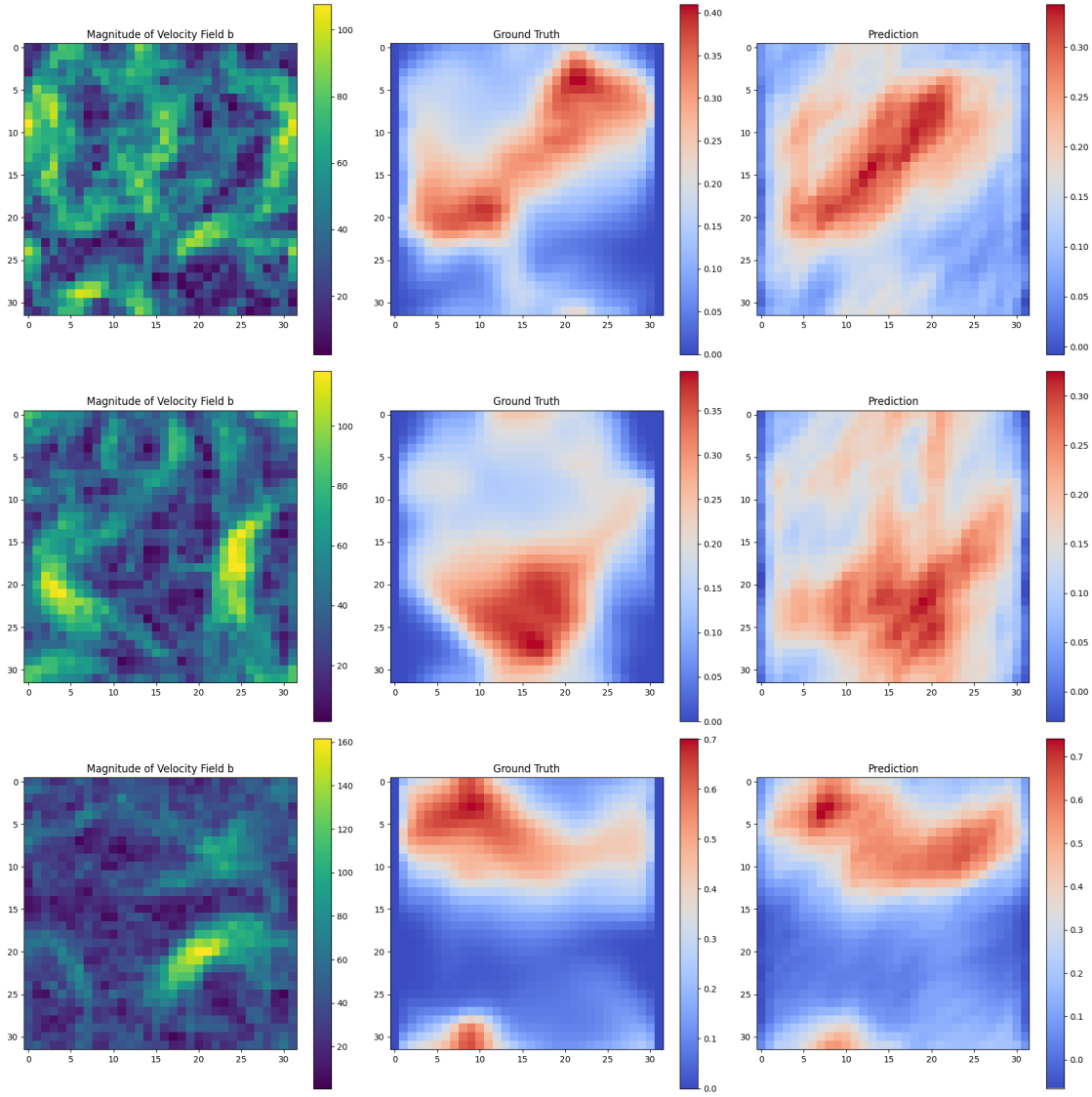

Figure 8: Predictions on a square domain at resolution 32×32 of PDE solutions from an FNO trained at the same resolution. Each row represents a different PDE example.

Table 2: Relative L_2 test error (11) on a curved domain for different data resolution grids: $s \times s$ for the training data and $s' \times s'$ for the testing data.

Resolutions	s' = 16	s' = 32	s' =63
$s = 16$	**0.56566**	0.611083	0.606691
$s = 32$	1.758391	**0.540448**	0.542250
$s = 63$	1.274045	0.537671	**0.534271**

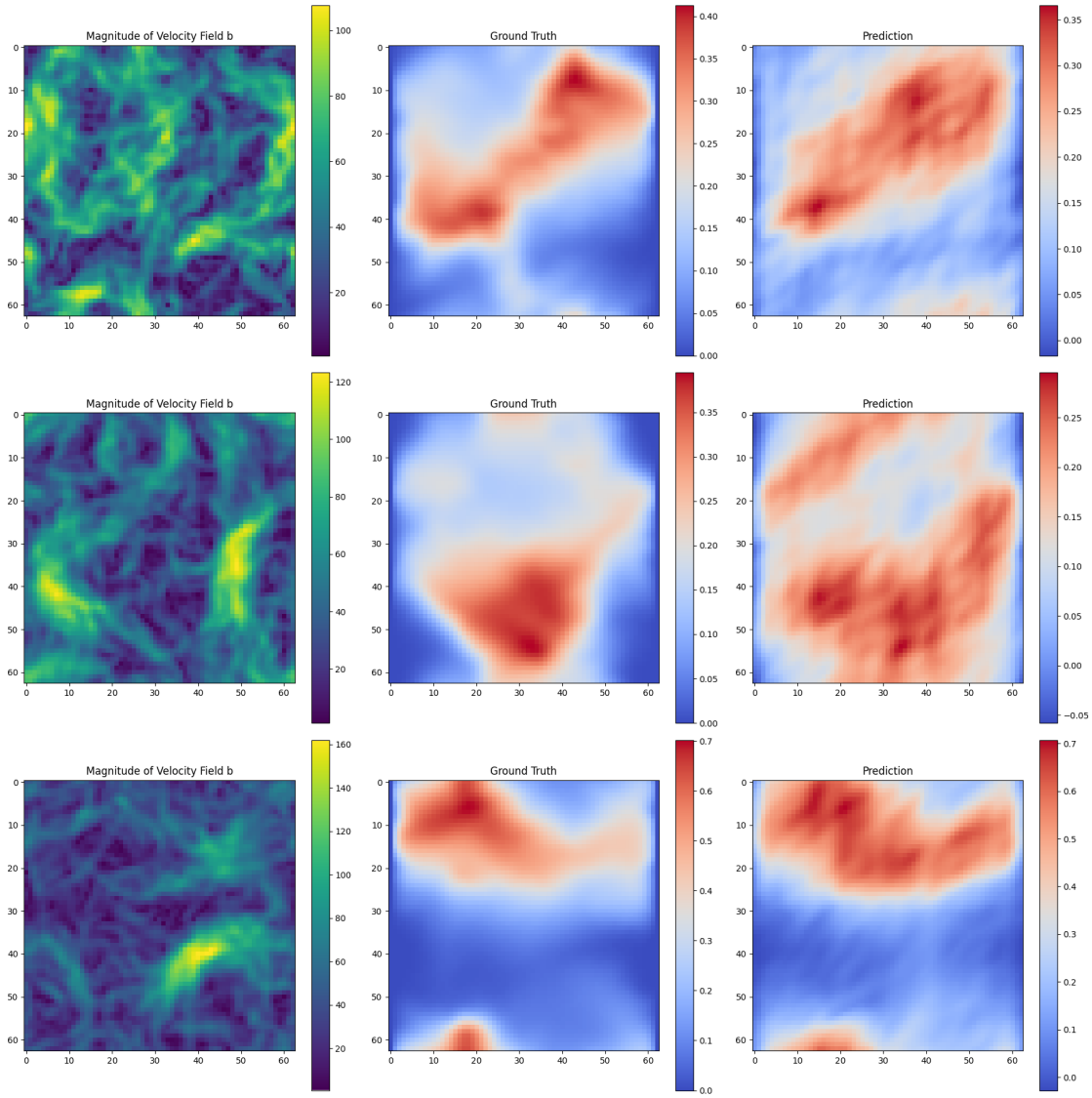

Figure 9: Predictions on a square domain at resolution 63×63 of PDE solutions from an FNO trained at the same resolution. Each row represents a different PDE example.

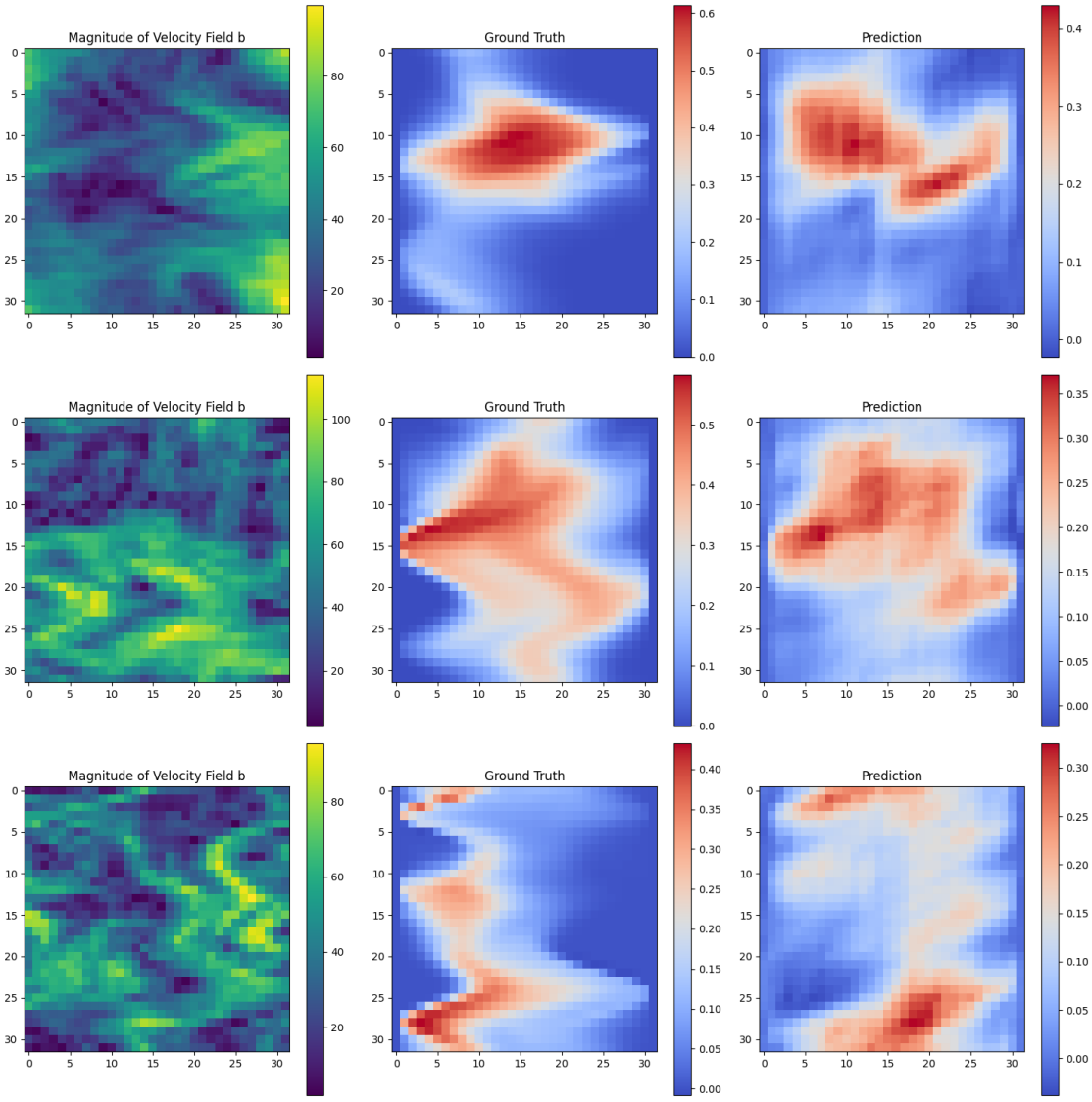

Figure 10: Predictions on a curved domain at resolution 32×32 of PDE solutions from an FNO trained at the same resolution. Each row represents a different PDE example.

6 Conclusion and Further Work to be Done

We have demonstrated that Fourier neural operators show promise at predicting the transition paths but are more accurate at predicting the solution on square domains than non-square domains. Better hyperparameter optimization might bridge the difference in accuracy between the two cases. Also if we understand how each hyperparameter contributes to the predicted solution (i.e transition path shape accuracy and probability accuracy) then this might inform better choices of hyperparameter that lower the error of predictions on non-square domains. The use of high performance computers to test over lots of hyperparameter combinations would be beneficial for training better FNOs that can be used in practical applications.

Although we hypothesised in Section 3.1 that a solution to equation (16) exists and is unique when b is L^2, there are reasons to believe this might not be the case. A lack of a smooth weak solution for u might explain why we are unable to reduce the relative L_2 test error further. For example in [1] the authors are able to get relative L_2 test errors of approximately 10^{-2} for predicting the coefficient to solution map of Darcy flows in 2-D, a similar but simpler problem. This is an order of 10 better than the FNOs achieved when applied to turbulence in this paper.

We list some follow-on research to be done.

- Operator learning the transition paths in a time range $[0, T]$ for a turbulent velocity field that changes in time. This situation requires larger training sets and requires a model of turbulence that comes from solving the Navier-Stokes equation so that there is a consistent turbulence field in time.
- Developing a proof for existence and if possible uniqueness of solutions to the PDEs that define the committor functions for non-Lipschitz drift coefficients.
- Trying other neural operator architectures to see if they have any advantage for this problem. For example multipole graph neural operators or DeepONet [13].
- Applying these techniques to domains without periodic boundaries (maybe modelling whole stretches of ocean for example).
- The predicted solutions from neural operators could be used as initial guesses in iterative schemes to speed up convergence to a more accurate solution.

Acknowledgements

Thank you to Martin Lotz and Tobias Grafke for suggesting this topic and for their useful discussion throughout the process. Thank you also to my family for their support and for listening to my ideas over the last 9 months.

7 References

[1] Nicolas Boullé and Alex Townsend. Chapter 3 - A mathematical guide to operator learning. In *Numerical Analysis Meets Machine Learning*, Siddhartha Mishra and Alex Townsend, editors, volume 25 of *Handbook of Numerical Analysis*, Elsevier, 2024, pages 83–125.

[2] Dustin Enyeart and Guang Lin. Some best practices in operator learning. arXiv (2412.06686), 2024.

[3] L.C. Evans. *Partial Differential Equations*. Graduate studies in mathematics. American Mathematical Society, 2010. ISBN 9780821849743.

[4] Crispin Gardiner. *Stochastic Methods: A Handbook for the Natural and Social Sciences*. 4th edition, Springer-Verlag, Berlin, Germany, 2009. ISBN 978-3-540-70712-7.

[5] Catherine F. Higham and Desmond J. Higham. Deep learning: An introduction for applied mathematicians. 2018.

[6] Desmond J. Higham and Peter E. Kloeden. *An Introduction to the Numerical Simulation of Stochastic Differential Equations*. 1st edition, Society for Industrial and Applied Mathematics, Philadelphia, PA, USA, 2021. ISBN 9781611976427.

[7] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. arXiv(1412.6980), 2014.

[8] A. N. Kolmogorov. The local structure of turbulence in incompressible viscous fluid for very large reynolds numbers. *Proceedings: Mathematical and Physical Sciences*, 434(1890):9–13, 1991.

[9] Nikola B. Kovachki, Samuel Lanthaler, and Andrew M. Stuart. Chapter 9 - operator learning: Algorithms and analysis. In *Numerical Analysis Meets Machine Learning*, Siddhartha Mishra and Alex Townsend, editors, volume 25 of *Handbook of Numerical Analysis*, Elsevier, 2024, pages 419–467.

[10] Robert H. Kraichnan. Small-scale structure of a scalar field convected by turbulence. *Physics of Fluids*, 11:945–953, 1968.

[11] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Graph kernel network for partial differential equations. arXiv(2003.03485), 2020.

[12] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. arXiv(2010.08895), 2021.

[13] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Animashree Anandkumar. Multipole graph neural operator for parametric partial differential equations. In *Advances in Neural Information Processing Systems (NeurIPS 2020)*, December 2020.

[14] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv preprint arXiv:1910.03193*, 2019.

[15] Philipp Metzner. *Transition Path Theory for Markov Processes.* Dissertation, The Free University of Berlin, 2008.

[16] Bernt Oksendal. *Stochastic Differential Equations: An Introduction with Applications.* Springer-Verlag, Berlin, Germany, 2003. ISBN 978-3-540-04758-2.

[17] Grigorios A. Pavliotis. *Stochastic Processes and Applications.* 1 edition, Springer-Verlag, New York, 2014. ISBN 978-1-4939-1322-0.

[18] Stephen B. Pope. *Turbulent Flows.* Cambridge University Press, 2000. ISBN 9780511840531.

[19] Catherine E. Powell, Gabriel Lord, and Tony Shardlow. *An Introduction to Computational Stochastic PDEs.* Texts in Applied Mathematics. 1 edition, Cambridge University Press, United Kingdom, August 2014. ISBN 9780521728522.

[20] Lewis Fry Richardson. *Weather Prediction by Numerical Process.* Cambridge Mathematical Library. Second edition, Cambridge University Press, 2007.

[21] Yixuan Sun, Ololade Sowunmi, Romain Egele, Sri Hari Krishna Narayanan, Luke Van Roekel, and Prasanna Balaprakash. Streamlining ocean dynamics modeling with fourier neural operators: A multiobjective hyperparameter and architecture optimization approach. *Mathematics*, 12(10), 2024.

8 Appendix - Derivation of Infinitesimal Generator

This section presents a derivation of the infinitesimal generator in (4)/ The content of this section is based on [16, p. 124].

Definition 8.1 (Infinitesimal generator)**.** Let $\{X_t\}$ be a time-homogeneous Itô diffusion in $\mathbb{R}^n$. The infinitesimal generator A of X_t is defined by

$$Af(x) = \lim_{t \downarrow 0} \frac{\mathbb{E}^x(f[X(t)]) - f(x)}{t}, \quad x \in \mathbb{R}^n.$$

The set of functions $f : \mathbb{R}^n \to \mathbb{R}$ such that the limit exists at x is denoted by $\mathcal{D}_A(x)$, while $\mathcal{D}_A$ denotes the set of functions for which the limit exists for all $x \in \mathbb{R}^n$.

The generator gives the rate of change of the expected value. We derive the infinitesimal generator L for equation (3) in one dimension. Note that here we use $\sigma = \sqrt{2}$. We use Taylor expansion to say that

$$\begin{aligned}\mathbb{E}^x[f(X_t)] &\approx \mathbb{E}^x[f(x) + f'(x)(X_t - X_0) + \frac{1}{2}f''(x)(X_t - X_0)^2] \\ &= f(x) + bf'(x)t + \frac{1}{2}f''(x) \times 2t\end{aligned}$$

where the last line uses the linearity of expectation, the mean displacement is $b \times t$, and the variance $2t$. Note $\mathbb{E}^x[\cdot]$ is the expectation conditional on $X_0 = x$. With this approximation we can now write that by definition

$$\begin{aligned}Lf(x) &= \lim_{t \downarrow 0} \frac{(bf'(x) - f''(x))t}{t} \\ &= bf'(x) - f''(x).\end{aligned}$$

It can be seen that the factor $\sqrt{2}$ in equation (1) acts as a normalizing constant for the infinitesimal operator. This proof can be made more rigorous and extended to higher dimensions by using the general Ito formula on $Y_t := f(X_t)$ (see [16, pg. 49]) to show that the linear operator has the form of equation (4).